COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	October 02, 2026	**DATE SUBMITTED:**	October 02, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

Activity 10.1 **Graphs**

1. PROCEDURE OUTPUT

ILO A

Output:

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Observations:
The output shows the level-wise output of Breadth First Search Traversal of the given weighted graph starting from vertex 1. The algorithm uses a queue-based FIFO data structure and visits neighbouring vertices systematically before reaching deeper vertices, resulting in the order of vertices visited 1 -> 2 -> 5 -> 4 -> 8 -> 3 -> 6 -> 7. The adjacency list is clearly showing the non-zero edge.

ILO B

Output:

BFS:

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

DFS:

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Observations:

The output demonstrates the DFS algorithm traversing the unweighted graph by exploring as far as possible along each branch and then backtracking. Unlike BFS, the traversal proceeds directly down path branches to visit vertices in the order 1 -> 5 -> 8 -> 6 -> 7 -> 3 -> 4 -> 2. The adjacency list

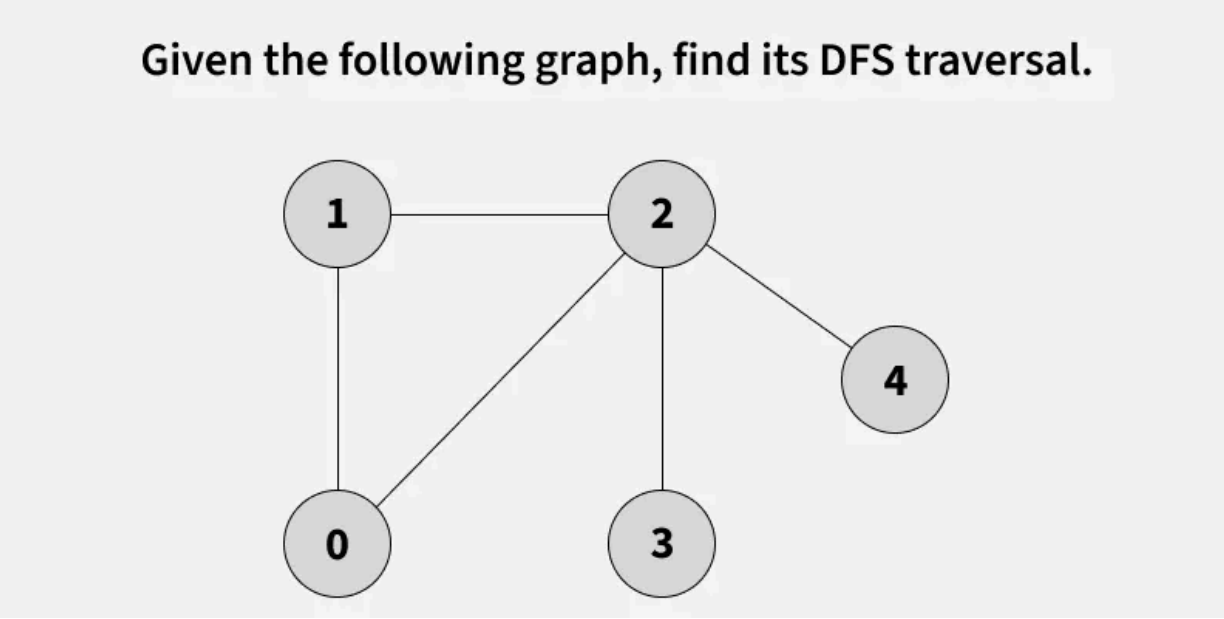
represents the vertices and the corresponding edge weights as zeros.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1.) A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

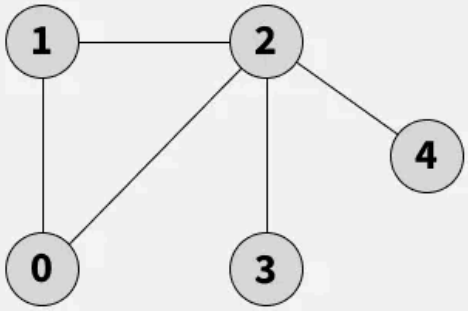
The most useful algorithm to do this is Depth-First Search (DFS). DFS starts on a tree branch and goes as far as it can on the branch and then backtracks to the previously explored nodes and follows the other paths. DFS is implemented as a stack (or recursion) of a "go deep first, then backtrack" traversal strategy, while Breadth-First Search (BFS) traverses level by level using a queue.

2.) Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.



01 Step

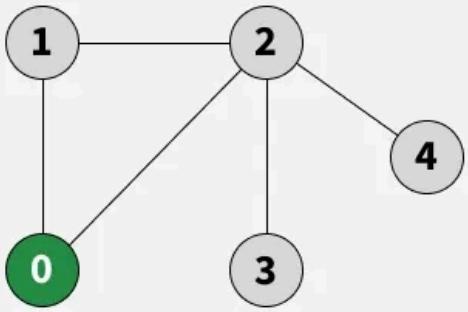
Maintain a visited array that tracks which nodes have been explored.



	0	1	2	3	4
visited[] =	F	F	F	F	F
res[] =					

02 Step

We start with source vertex 0.

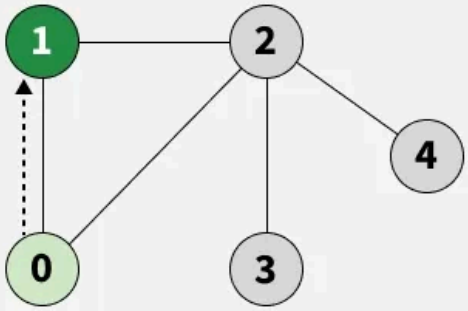


	0	1	2	3	4
visited[] =	T	F	F	F	F
res[] =	0				

dfsRec (0)

03 Step

We visit the neighbour of 0, i.e., 1.

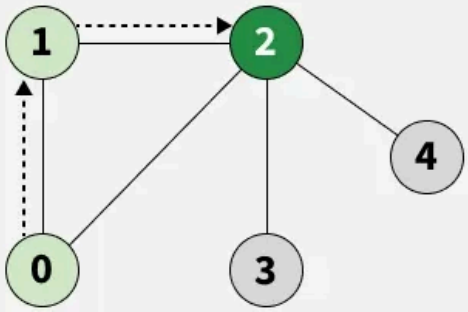


	0	1	2	3	4
visited[] =	T	T	F	F	F
res[] =	0	1			

dfsRec (1)
dfsRec (0)

04 Step

We visit the neighbour of 1, i.e., 2.

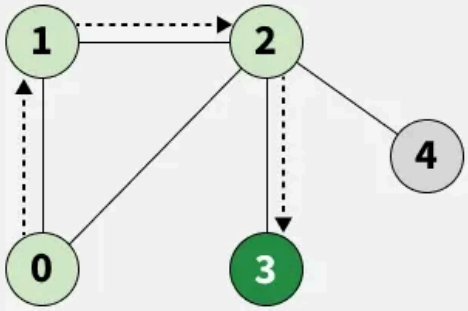


	0	1	2	3	4
visited[] =	T	T	T	F	F
res[] =	0	1	2		

dfsRec (2)
dfsRec (1)
dfsRec (0)

05 Step

We visit the neighbour of 2, i.e., 3.

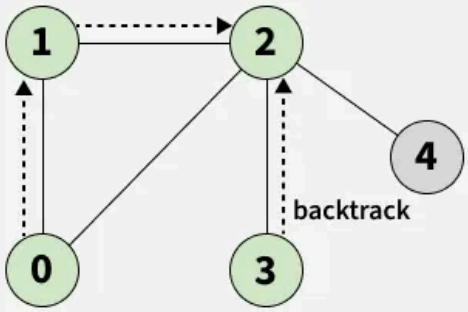


	0	1	2	3	4
visited[] =	T	T	T	T	F
res[] =	0	1	2	3	

dfsRec (3)
dfsRec (2)
dfsRec (1)
dfsRec (0)

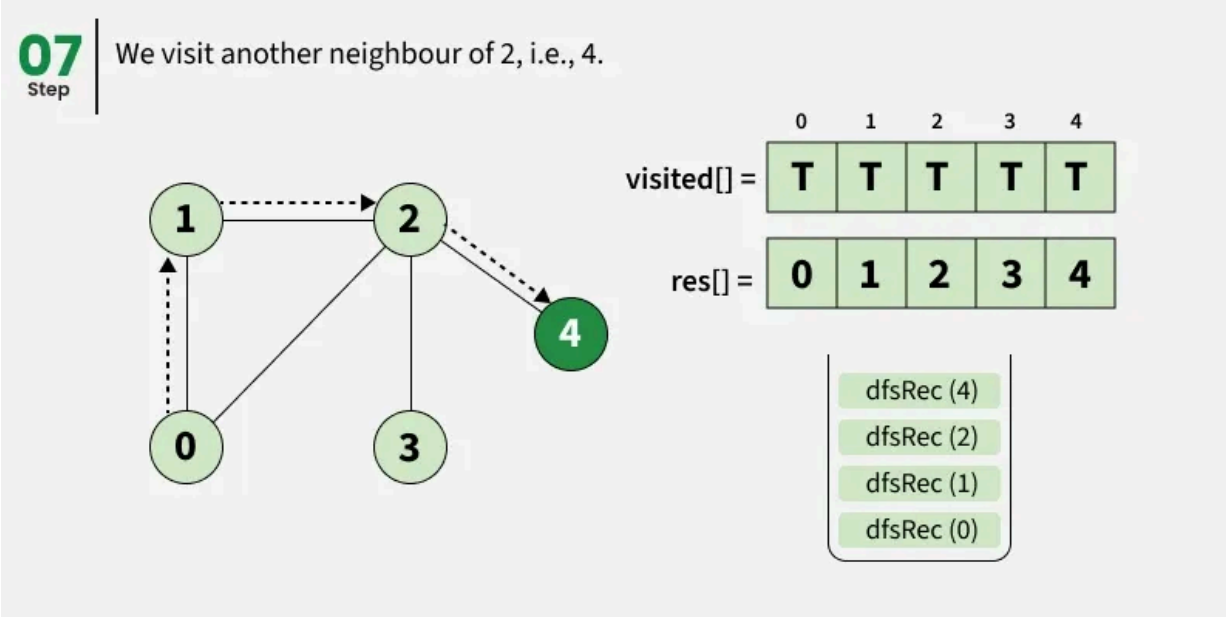
06 Step

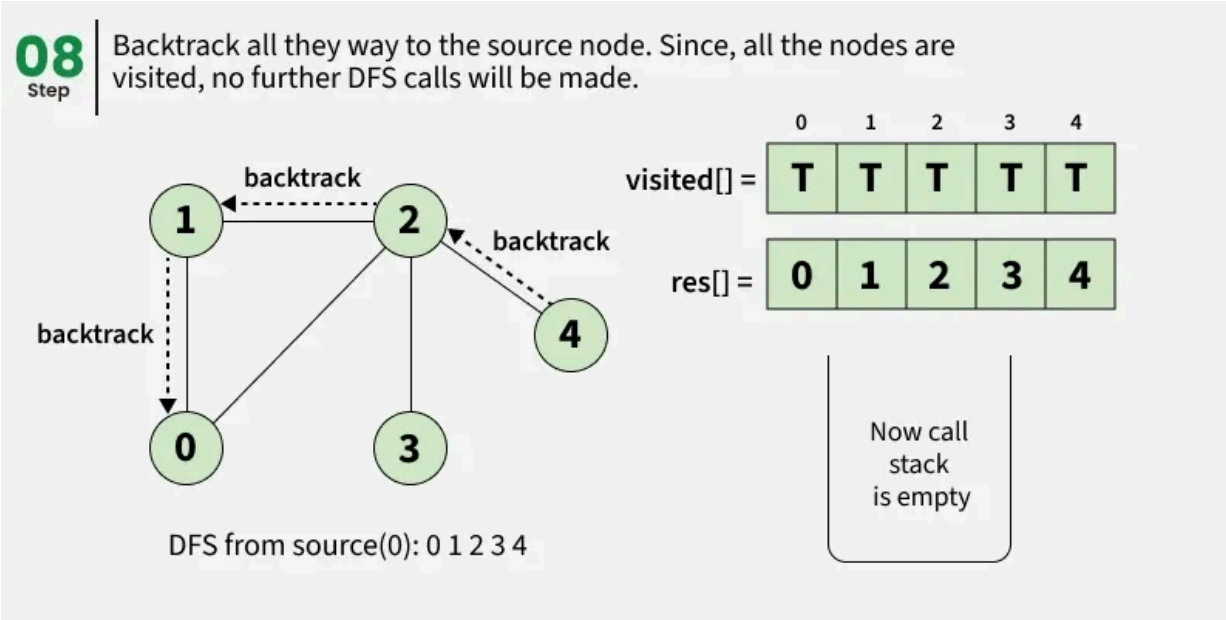
Backtracking from 3.



	0	1	2	3	4
visited[] =	T	T	T	T	F
res[] =	0	1	2	3	

dfsRec (2)
dfsRec (1)
dfsRec (0)





Link of reference: https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/

3.) In the performed code, what data structure is used to implement the Breadth First Search?

In the actual code, the frontier of vertices to explore will be implemented as a Queue (std::queue<size_t>) that is used in First-In, First-Out (FIFO) fashion. Further, a Set (std::set<size_t>) is used to remember vertices that are visited.

4.) How many times can a node be visited in the BFS?

A node is added to the "Visited Set" and processed only once. The visited check (visited.find(current_vertex) != visited.end()) means that the contents and outgoing connections of a vertex will be executed only once, although the vertex can be pushed onto a queue or checked as a neighbor by incoming edges many times.

Conclusion:

This activity showed how to implement and traverse graph data structures in C++ with the help of adjacency lists. By using the Breadth-First Search (BFS), we discussed about the dynamics of level order traversal with the help of FIFO queue (std::queue) and std::set to track visited nodes so that every vertex is visited only once and infinite cycle is avoided. In addition, the similarities and differences between graph algorithms (e.g., Depth-First Search, DFS) and classic tree traversals illustrated the importance of structural properties in determining data structures and algorithms. The correct class scope and sequence bounds to use in combination with the use of a pointer binding and a loop boundary are essential when dealing with a custom template data structure, as demonstrated in the debugging of template declarations, pointer bindings, and loop boundaries.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.